Incident Response Runbook

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This runbook describes how the team runs incident response across the platform (api-service, auth-service, worker-service, ingestion-service, gateway, scheduler). It is a general overview of the lifecycle — detect, triage, mitigate, communicate, and learn — not a fix for any single failure. For specific classes of failure, defer to the topical handbook pages and the shared incident/bug database, which records every past incident with a symptom and a resolution so responders can pattern-match quickly.

Standards

Roles and severity

Every incident has one Incident Commander (IC) who owns coordination and decisions, plus an Ops lead who drives mitigation and a Comms lead who owns updates. Assign a severity up front and reassess as you learn:

- **SEV1** — broad outage or data loss/exposure; page immediately, all hands.
- **SEV2** — major degradation or a single critical service down; page on-call.
- **SEV3** — contained or partial impact with a workaround; handle in business hours.

Detect and triage

Alerts come from SLO burn-rate, error-rate, and latency monitors, plus readiness probes that verify real dependencies rather than mere process liveness. On page, the IC establishes a single incident channel, confirms scope (which services, which regions, blast radius), and forms a first hypothesis.

```
# Triage starting points
# 1. What changed? recent deploy, config, cert, or quota change
# 2. Correlate logs across services by the traceparent correlation id
# 3. Search the bug database for the symptom string
SELECT bug_id, title, resolution
FROM known_bugs
WHERE LOWER(title) LIKE '%certificate has expired%';
```

Mitigate before you root-cause

Restore service first; a full diagnosis can wait. Prefer the fastest safe lever:

- Roll back the most recent revision if the timeline points at a deploy.
- Shed or throttle load; open a circuit breaker to stop retry amplification.
- Scale out, raise a quota, or fail over to a healthy region.
- Confirm the fix with the same signals that fired the alert before declaring recovery.

Communicate

The Comms lead posts a short status at declaration and then at a fixed cadence (e.g. every 30 minutes for SEV1/2): current impact, what is being tried, and the next update time. Keep internal timeline notes as you go — they become the postmortem.

Postmortem

Within a few business days, write a blameless postmortem: timeline, impact, contributing factors, what went well/poorly, and concrete action items with owners. File the incident into the bug database with a clear symptom and resolution so the next responder finds it. Track action items to completion.

Common failures and fixes

- Symptom search returns a known match in the bug database → apply the recorded resolution directly instead of re-diagnosing (e.g. expired outbound TLS cert, BUG-1065).
- A brief downstream blip escalated into a full outage → open the circuit breaker and enforce backoff-with-jitter to stop retry amplification (BUG-1067).
- A deploy caused 403 PERMISSION_DENIED across a service → roll back, then grant the missing least-privilege IAM role and redeploy (BUG-1017).
- Cross-service failure is hard to trace during triage → ensure a traceparent correlation id is propagated so logs join across services (BUG-1073).
- Load balancer keeps routing to a broken instance → fix the readiness check to verify real dependencies so degraded instances drain (BUG-1075).

Checklist

- IC, Ops, and Comms roles assigned; severity set and reassessed.
- Single incident channel open; scope and blast radius confirmed.
- "What changed?" checked (deploy, config, cert, quota).
- Bug database searched for the symptom before deep debugging.
- Mitigation applied and verified before root-causing.
- Status updates posted on a fixed cadence with a next-update time.
- Blameless postmortem written; incident filed to the bug database; action items owned.

Related incidents

BUG-1065, BUG-1067, BUG-1017, BUG-1073, BUG-1075